

GRAPH THEORY & ITS APPLICATIONS

World of Graphs

*An Introduction to Graph Theory
and Its Applications*

MADE BY

Chris Abraham Blessan · Aben Vargheese

TABLE OF CONTENTS

FRONT MATTER	2
Acknowledgements	
FRONT MATTER	3
Preface	
UNIT ONE	6
Introduction to Graphs	
UNIT TWO	10
Connectivity & Traversability in Graphs	
UNIT THREE	15
Matchings, Decompositions & Planarity	
REFERENCE	18
References	
CLOSING	20
About the Authors	

Acknowledgements

We are grateful to our teachers, mentors, and peers whose guidance shaped both our understanding of graph theory and our approach to teaching it. Their feedback on early drafts of this material helped clarify explanations that might otherwise have remained abstract.

We also thank the students whose questions, over many semesters, revealed exactly where a textbook needed to slow down, add an example, or state a definition more carefully. This book is, in large part, a response to those questions.

Finally, we thank our families for their patience and support throughout the writing of this book.

Preface

Graphs are among the simplest and most powerful structures in mathematics. A handful of dots and lines can model a road network, a social circle, a molecule, or the internet itself. Yet beneath this simplicity lies a rich and rigorous theory that has shaped modern computer science, operations research, and applied mathematics.

World of Graphs is written as a concise companion for students encountering graph theory for the first time. It follows a structured progression: beginning with the language of graphs, moving through connectivity and traversability, and concluding with trees, matchings, and planarity.

Each section defines its concepts precisely, illustrates them with worked examples, and states the key theorems a learner is expected to know and apply. The aim throughout is clarity without sacrificing mathematical rigor, so that the material serves equally well for classroom instruction and self-study.

We hope this book helps you see, quite literally, the world in graphs.

UNIT ONE

Introduction to Graphs

1.1 What is a Graph?

A **graph** G is a mathematical structure used to model pairwise relationships between objects. Formally, a graph is an ordered pair $G = (V, E)$, where V is a finite non-empty set called the vertex set, and E is a set of unordered pairs of vertices called the edge set.

The elements of V are called vertices (or nodes), and the elements of E are called edges (or lines). An edge $e = \{u, v\}$ is said to join vertices u and v , and u, v are called the end vertices of e .

Order and Size. The order of a graph G is the number of vertices, denoted $|V(G)|$ or n . The size of G is the number of edges, denoted $|E(G)|$ or m .

Graphs are typically drawn with points representing vertices and line segments or curves representing edges. The physical layout of a drawing has no mathematical significance — what matters is which vertices are joined by which edges.

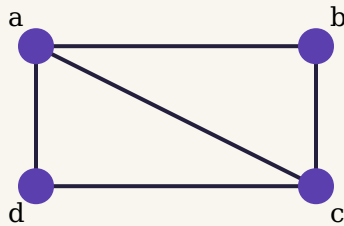


Fig 1.1 — A simple graph with 4 vertices and 5 edges

Graph theory traces its origin to Leonhard Euler's 1736 solution of the *Königsberg bridge problem*, widely regarded as the first paper in the subject. Since then, the field has grown to underpin network design, algorithm analysis, chemistry, linguistics, and countless areas of applied mathematics and computer science.

1.2 Basic Graph Terminology

Adjacent Vertices. Two vertices u and v are adjacent if there is an edge joining them, i.e., $\{u, v\}$ is in E .

Adjacent Edges. Two edges are adjacent if they share a common end vertex.

Incidence. An edge $e = \{u, v\}$ is incident with each of its end vertices u and v .

Loop. An edge joining a vertex to itself is called a loop.

Multiple Edges. Two or more edges joining the same pair of vertices are called multiple or parallel edges.

Simple Graph. A graph with no loops and no multiple edges is called a simple graph. Unless stated otherwise, graphs in this book are assumed to be simple.

Empty Graph. A graph with $E = \emptyset$ (no edges) is called an empty graph.

Null Graph. A graph with $V = \emptyset$ (no vertices, and hence no edges) is called a null graph.

Trivial Graph. A graph with exactly one vertex and no edges is called a trivial graph.

1.3 Types and Classes of Graphs

Complete Graph. A simple graph in which every pair of distinct vertices is adjacent is called a complete graph, denoted K_n for n vertices. K_n has $n(n-1)/2$ edges.

Regular Graph. A graph in which every vertex has the same degree is called a regular graph. If every vertex has degree r , the graph is r -regular.

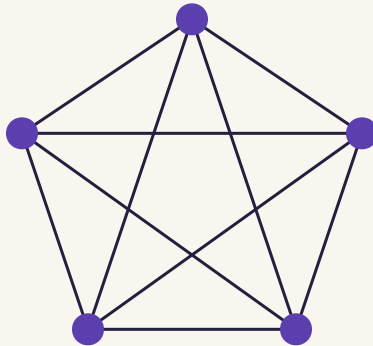


Fig 1.2 — The complete graph K_5 (every pair of vertices adjacent)

Bipartite Graph. A graph G is bipartite if $V(G)$ can be partitioned into two disjoint sets V_1 and V_2 such that every edge joins a vertex in V_1 to a vertex in V_2 . If every vertex of V_1 is joined to every vertex of V_2 , G is a complete bipartite graph, denoted $K_{\{m,n\}}$.

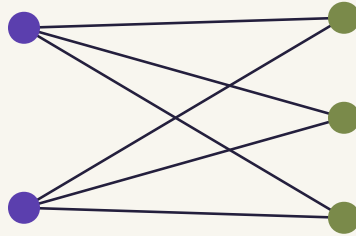


Fig 1.3 — The complete bipartite graph $K_{\{2,3\}}$

Subgraph. A graph H is a subgraph of G if $V(H) \subseteq V(G)$ and $E(H) \subseteq E(G)$.

Spanning Subgraph. A subgraph H of G is a spanning subgraph if $V(H) = V(G)$, i.e., it contains every vertex of G .

Complement of a Graph. The complement of a simple graph G , denoted $\bar{G}$, has the same vertex set as G , with two vertices adjacent in $\bar{G}$ if and only if they are not adjacent in G .

Connected Graph. A graph is connected if there is a path between every pair of vertices. A graph that is not connected is called disconnected, and consists of two or more connected components.

1.4 Degree Sequences and the Handshaking Lemma

Degree. The degree of a vertex v , denoted $\deg(v)$, is the number of edges incident with v (a loop counting twice).

Degree Sequence. The degree sequence of a graph is the list of vertex degrees, usually arranged in non-increasing order.

Theorem 1.1 (Handshaking Lemma). For any graph $G = (V, E)$, the sum of the degrees of all vertices equals twice the number of edges: $\sum \deg(v) = 2|E|$

Corollary. Every graph has an even number of vertices of odd degree.

Example. Consider a graph with degree sequence (3, 3, 2, 2, 2). The sum is 12, so the graph has 6 edges. Since there are exactly two vertices of odd degree, this is consistent with the corollary above.

1.5 Graph Representation: Matrices

Adjacency Matrix. For a graph G with vertices $v_1, \dots, v_n$, the adjacency matrix A is an $n \times n$ matrix where $A[i][j] = 1$ if v_i and v_j are adjacent, and 0 otherwise. For a simple graph, A is symmetric with zero diagonal.

Incidence Matrix. The incidence matrix B of a graph with n vertices and m edges is an $n \times m$ matrix where $B[i][j] = 1$ if vertex v_i is incident with edge e_j , and 0 otherwise.

Matrix representations are the foundation of computational graph algorithms: adjacency matrices allow constant-time edge lookups, while adjacency lists (a related structure not detailed here) are preferred for sparse graphs due to lower memory use.

1.6 Isomorphic Graphs

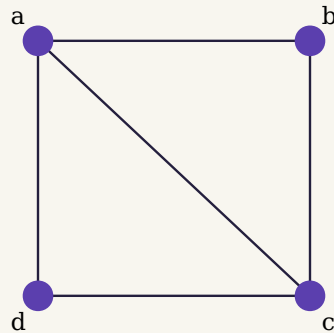
Graph Isomorphism. Two graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are isomorphic if there exists a bijection $f: V_1 \rightarrow V_2$ such that $\{u, v\} \in E_1$ if and only if $\{f(u), f(v)\} \in E_2$.

Isomorphic graphs are structurally identical, differing only in the labeling of vertices. Necessary (but not sufficient) conditions for isomorphism include equal order, equal size, and identical degree sequences. Determining isomorphism in general is computationally challenging and remains an active area of research.

Bipartite graphs model matching problems such as assigning students to schools or workers to tasks. Regular graphs appear in the design of interconnection networks, where a uniform degree ensures balanced load across all nodes. The adjacency matrix representation, in particular, is the starting point for spectral graph theory, which studies graph properties through the eigenvalues of associated matrices — a technique used in Google's original PageRank algorithm and in modern graph neural networks.

Worked Examples

Example 1.1. Let G be a graph with vertex set $\{a, b, c, d\}$ and edge set $\{ab, bc, cd, da, ac\}$. This graph has order 4 and size 5. Vertex a has degree 3 (adjacent to b, c , and d), so a and c (also degree 3) are the highest-degree vertices, while b and d have degree 2 each. The degree sequence is $(3, 3, 2, 2)$, and by the handshaking lemma the sum 10 correctly equals $2|E| = 10$.



The graph of Example 1.1

Example 1.2. The Petersen graph, a well-known 3-regular graph on 10 vertices and 15 edges, is often used to illustrate subtle properties in graph theory: it is non-planar, has no Hamiltonian cycle, yet is highly symmetric and vertex-transitive. It frequently appears as a counterexample to plausible-looking conjectures.

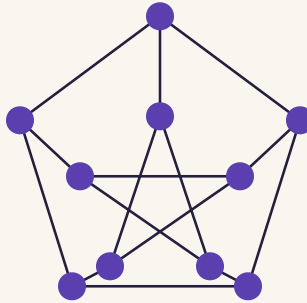


Fig 1.4 — The Petersen graph: 10 vertices, 15 edges, 3-regular

Review Questions

- Define a simple graph and explain how it differs from a multigraph.
- Prove that the sum of degrees in any graph is even.
- Construct the complement of the complete bipartite graph $K_{\{2,3\}}$. What does it look like?
- Give an example of two non-isomorphic graphs with identical degree sequences.
- Write the incidence matrix for the graph in Example 1.1.

Connectivity and Traversability in Graphs

2.1 Bridges and Cut-Vertices

Bridge. An edge whose removal increases the number of connected components of a graph is called a bridge (or cut-edge).

Cut-Vertex. A vertex whose removal (together with its incident edges) increases the number of connected components is called a cut-vertex.

Theorem 2.1. An edge e of a connected graph G is a bridge if and only if e lies on no cycle of G .

Bridges and cut-vertices identify the structurally fragile parts of a network — points whose failure disconnects the system. This idea is central to the analysis of network reliability.

2.2 Trees, Rooted Trees, and Binary Trees

Tree. A connected acyclic graph is called a tree. A tree on n vertices always has exactly $n - 1$ edges.

Forest. A graph in which every component is a tree (i.e., an acyclic graph, not necessarily connected) is called a forest.

Leaf. A vertex of degree 1 in a tree is called a leaf, or pendant vertex.

Rooted Tree. A tree in which one vertex is distinguished as the root is called a rooted tree. Every other vertex has a unique path to the root, inducing a parent–child hierarchy.

Binary Tree. A rooted tree in which every vertex has at most two children is called a binary tree.

Theorem 2.2. A connected graph on n vertices is a tree if and only if it has exactly $n - 1$ edges and contains no cycle.

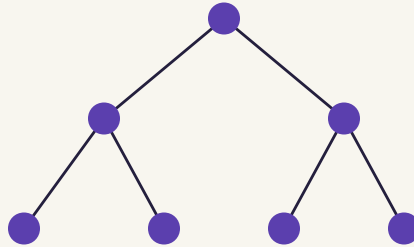


Fig 2.1 — A rooted binary tree

2.3 Spanning Trees and Minimum Spanning Trees

Spanning Tree. A spanning tree of a connected graph G is a spanning subgraph of G that is also a tree — it includes every vertex of G using the minimum number of edges required to keep the graph connected.

Minimum Spanning Tree (MST). If each edge of G is assigned a weight, a minimum spanning tree is a spanning tree whose total edge weight is as small as possible. Two classical greedy algorithms construct an MST:

- **Kruskal's Algorithm** — repeatedly add the cheapest edge that does not create a cycle, until $n - 1$ edges are chosen.
- **Prim's Algorithm** — grow a single tree from an arbitrary starting vertex, at each step adding the cheapest edge connecting the tree to a new vertex.

Both algorithms are proven optimal by the cut property: for any partition of vertices into two sets, the minimum-weight edge crossing the partition belongs to some MST.

Kruskal's Algorithm (outline):

1. Sort all edges of G in non-decreasing order of weight.
2. Initialize an empty edge set T .
3. For each edge in sorted order, add it to T if doing so does not create a cycle among the edges already in T .
4. Stop when T contains $n - 1$ edges; T is the minimum spanning tree.

Prim's Algorithm (outline):

1. Choose an arbitrary starting vertex and mark it as visited.
2. Among all edges with exactly one visited end vertex, select the one of minimum weight.
3. Add this edge and its unvisited end vertex to the tree; mark the vertex visited.
4. Repeat until all vertices are visited.

2.4 Blocks and Block-Cut Tree Representation

Block. A block of a graph is a maximal connected subgraph with no cut-vertex of its own — either a maximal 2-connected subgraph, a bridge, or an isolated vertex.

Block-Cut Tree. The block-cut tree of a graph represents the arrangement of its blocks and cut-vertices as a tree, with an edge joining a block-node to a cut-vertex node whenever the cut-vertex belongs to that block.

This representation compresses a complex graph into a tree structure that reveals precisely how its 2-connected pieces are stitched together through cut-vertices — useful in analyzing the modular structure of large networks.

2.5 Eulerian Graphs

Eulerian Circuit. A closed walk that traverses every edge of a graph exactly once is called an Eulerian circuit. A graph containing such a circuit is called Eulerian.

Eulerian Trail. An open walk traversing every edge exactly once (but not returning to the start) is called an Eulerian trail.

Theorem 2.3 (Euler, 1736). A connected graph G is Eulerian if and only if every vertex of G has even degree.

Theorem 2.4. A connected graph G has an Eulerian trail (but not a circuit) if and only if exactly two vertices have odd degree.

These results resolved the Königsberg bridge problem: since all four landmasses had odd degree, no walk crossing each bridge exactly once was possible.

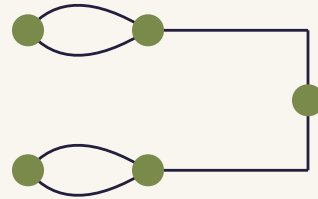


Fig 2.2 — The Königsberg bridges, modeled as a graph

2.6 Hamiltonian Graphs

Hamiltonian Cycle. A cycle that visits every vertex of a graph exactly once (returning to the start) is called a Hamiltonian cycle. A graph containing one is Hamiltonian.

Hamiltonian Path. A path visiting every vertex exactly once, without returning to the start, is a Hamiltonian path.

Unlike the Eulerian case, no known necessary and sufficient condition characterizes Hamiltonian graphs in general — determining Hamiltonicity is an NP-complete problem. However, useful sufficient conditions exist:

Theorem 2.5 (Dirac's Theorem, 1952). If G is a simple graph with $n \geq 3$ vertices and every vertex has degree at least $n/2$, then G is Hamiltonian.

Relationship between Eulerian and Hamiltonian Graphs. The two properties are logically independent: a graph may be Eulerian without being Hamiltonian, Hamiltonian without being Eulerian, both, or neither. Eulerian graphs traverse every edge once; Hamiltonian graphs visit every vertex once — these are fundamentally different combinatorial requirements.

2.7 Digraphs

Directed Graph (Digraph). A digraph $D = (V, A)$ consists of a vertex set V and a set A of ordered pairs of vertices called arcs (or directed edges). An arc (u, v) is directed from u to v .

In-degree and Out-degree. For a vertex v in a digraph, the in-degree is the number of arcs directed into v , and the out-degree is the number of arcs directed out of v .

Digraphs model relationships with inherent direction — dependencies, one-way links, precedence in scheduling — and extend naturally to concepts such as directed paths, strong connectivity, and tournaments, in which every pair of vertices is joined by exactly one arc.

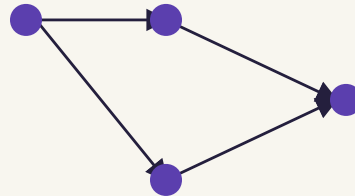


Fig 2.3 — A directed acyclic graph (DAG)

Applications

Spanning trees are the backbone of network design: telecommunication and electrical grid planners use minimum spanning trees to connect every location at the lowest possible infrastructure cost. Kruskal's and Prim's algorithms, though simple, remain in active industrial use for exactly this reason.

Eulerian circuits solve route-inspection problems — a postal worker, a street-sweeping truck, or a drone inspecting power lines all want to traverse every required edge exactly once with minimum repeated travel. Hamiltonian paths, by contrast, underlie the famous Traveling Salesman Problem, in which a route must visit every city exactly once at minimum total cost; unlike the Eulerian case, no efficient general algorithm is known, making it a cornerstone problem in computational complexity theory.

Digraphs and their in/out-degree structure model task scheduling through directed acyclic graphs (DAGs), where an arc (u, v) means task u must precede task v . Topological sorting of a DAG produces a valid task order — a technique used in build systems, spreadsheet recalculation, and course-prerequisite planning.

About the Authors

Chris Abraham Blessan and **Aben Vargheese** bring together academic training in mathematics and computer science to present graph theory in a form that is accessible without sacrificing rigor.

World of Graphs grew out of classroom teaching and mentoring, aiming to connect formal definitions to intuitive, worked examples for the next generation of learners.

WORLD OF GRAPHS